

Modern LLVM-based Compiler Autotuning for WCET Optimization

Technical Report

G. Magnani, D. Baroffio, F. Reghenzani
G. Agosta, W. Fornaciari

The 46th IEEE Real-Time Systems Symposium
December 2–5, 2025
Boston, MA, USA

1 Introduction

This technical report gathers the links to the data of the Lavinium exploration on the Malardalen WCET and Polybench benchmark suites, and the raw execution times gathered on the RISC-V evaluation board. Moreover, we also provide further information on the pass sequence analysis and validation we found for Lavinium.

2 Raw analysis data

The raw analysis data can be found here:
<https://github.com/ilgeco/RTSS-Lavinium-RAW-data.git>

3 Raw RISC-V execution time data

The raw execution time data captured on the RISC-V MCU can be found here:
<https://github.com/ilgeco/RTSS-Lavinium-RAW-data.git>

4 Pass sequences description

We analyzed the data reported in Section 2 to gain some knowledge about the best sequences and passes: i.e., the ones that have the greatest probability of being in the sequence that minimizes WCET.

For each benchmark and exploration policy, we have gathered the sequences of optimizations that minimized the WCET function-wise. The ten most frequent passes present in the optimization sequence can be grouped principally into two categories:

- The first group of passes consists of simplification passes that remove redundant operations or combine multiple instructions into simpler ones. This group contains:
 - **gvn**. This pass performs Global Value Numbering (GVN) to eliminate fully redundant instructions. In general, value numbering determines when two computations in a program are equivalent and eliminates one of them with a semantics-preserving optimization. LLVM’s GVN also performs simple dead load elimination, which removes load instructions that are not used by any other instruction.
 - **sroa**. This pass performs Scalar Replacement Of Aggregates (SROA). SROA breaks up IR-level memory allocations of aggregate types (e.g., structs or arrays) into individual scalar allocations, one for each of the struct’s members, if possible. Then, it builds the primary LLVM Static Single-Assignment (SSA) representation using the new individual allocations.
 - **instcombine**. This pass combines instructions to form fewer, simpler instructions, without modifying the CFG. It is based on a pattern-matching algorithm that tries to identify instruction patterns to rewrite them into simpler forms.
 - **mldst-motion**. Given a so-called diamond (also known as a hammock) – i.e., a CFG structure where there is a single entry and a single exit block, with two alternative paths between them – this pass performs Merged Load Store (MLDST)-Motion. MLDST-motion moves redundant load and store instructions that exist in both alternative paths before and after the diamond, respectively. This optimization aims to hide load latencies, trigger so-called if-conversion, and reduce code size.
- The second group are passes that are loop-related and tend to remove or simplify the instructions inside the loop. This group includes:
 - **licm**. This pass performs Loop-Invariant Code Motion (LICM). LICM attempts to remove as much code as possible from the body of a loop. It does this by moving code either into the loop’s preheader block or into the loop’s exit blocks.
 - **indvars**. This loop transformation acts on loop induction variables, i.e., variables that determine the loop count and, therefore, the termination condition of iterative loops. This is a loop normalization pass that transforms loops into simpler forms, suitable for subsequent analyses and transformations.

- **loop-deletion.** This pass is responsible for eliminating loops with non-infinite computable trip counts. A loop is eliminated if it does not have side effects (e.g., volatile instructions) and does not contribute to the computation of the function’s return value.
- **loop-vectorize.** The loop vectorizer merges consecutive iterations of a loop into one ‘wide’ iteration that utilizes SIMD instructions. As a result of this transformation, the index is also increased by the width of the SIMD vector rather than by one.
- **loop-unroll.** This pass implements the well-known loop-unroll technique. The loop-unroll replicates the body of a loop multiple times to reduce loop-control overhead and expose further optimization opportunities. It also adjusts the loop termination condition accordingly.
- **loop-idiom.** This pass implements an idiom recognizer that transforms simple loops into a non-loop form. It can recognize the patterns of `memset`, `memcpy`, `memmove`, and `strlen`, and substitute the loop with a call to the standard library.

Upon analyzing the subsequences of length greater than one, we found that their frequency is less than 2%. This provides strong evidence that there are no subsequences of two or more passes that are consistently beneficial to the WCET. Therefore, it is necessary to employ an exploratory approach on a case-by-case basis.